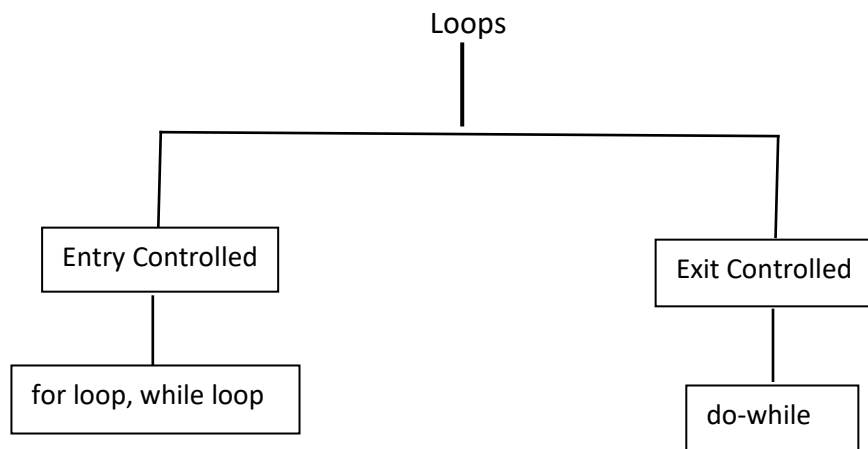


Iterations (loops)

Author: *Subhadev Das*

A loop in Java is a control flow statement that allows code to be executed repeatedly based on a condition. Each type of loop executes a block of code multiple times until a specified condition is met. This allows for repetitive tasks to be automated efficiently.



Entry Controlled Loops: Loops in which condition is checked first and code is executed only if condition is true are called Entry Controlled Loops. Example: for loop and while loop

Exit Controlled Loops: In Exit controlled loops condition is checked after executing the code. here the loops runs at least once even if the condition is false. Example: do while loop

** In this document we will discuss about for loop.

for loop Syntax:

```
for(initialization; condition; update statement){  
    //code  
}
```

Example 1:

```
for(int i=1; i<=5; i++){  
    System.out.print(i);  
}
```

Output: 1 2 3 4 5

Example 2:

```
for(int i=1; i<=10; i+=2){  
    System.out.print(i);  
}
```

Output: 1 3 5 7 9

Example 3:

```
for(int i=10; i>=1; i--){  
    System.out.println(i);  
}
```

Output: 10 9 8 7 6 5 4 3 2 1

Example 3:

```
for(double i=0.0; i<=2.0; i+=  
    0.5){  
    System.out.print(i);  
}
```

Output: 0.0 0.5 1.0 1.5 2.0

To find how many times the loop runs:

```
for (int i=1; i<=10; i++){  
    System.out.println(i);  
}
```

We can clearly see that the loop runs for 10 times

We can also find this using a formula:

```
for(int i=a; i<=b; i+=k){  
}
```

$$\text{Number of iterations} = \left\lfloor \frac{b-a}{k} \right\rfloor + 1$$

Applying this formula to the loop above :

$$\text{Number of iterations} = \left\lfloor \frac{10-1}{1} \right\rfloor + 1 = 10$$

Jump Statements: There are three jump statements: break, continue and return

break: the keyword “break” terminates the loop.

```
for(int i=1; i<=10; i++){  
    if(i==5)break;  
    System.out.println(i);  
}
```

Output: 1 2 3 4

here the loop breaks at i=5

continue: the keyword “continue” transfers the program control to next iteration skipping any code residing below the continue statement.

```
for(int i=1; i<=10; i++){  
    if(i==5)continue;  
    System.out.println(i);  
}
```

Output: 1 2 3 4 6 7 8 9 10

here the loop skips printing at i=5

Sequence

A sequence is an ordered list of that follow a specific rule or pattern.

Example 1: 1 2 3 4 5

Example 2: 1 3 5 7 9

We can print a sequence with for loop.

Example 1:

1 2 4 8 16 32 64

2^0 2^1 2^2 2^3 2^4 2^5 2^6

The term looks like: 2^i [0,6]

```
for( int i= 0; i<=6; i++){  
double k= Math.pow(2,i);  
System.out.println(k);  
}
```

**** For sequence, We must write the printing code inside the loop block.**

Example 2:

2 9 28 65 126

1^3+1 2^3+1 3^3+1 4^3+1 5^3+1

the term looks like: i^3+1 [1,5]

```
for(int i=1; i<=5; i++){  
double k= Math.pow(i,3)+1;  
System.out.println(k);  
}
```

Series

A series is the sum of the terms of a sequence.

Example 1: $S = 1 + 2 + 3 + 4 + 5$

Example 2: $S = \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6} + \frac{1}{7}$

We can find S (sum of sequence or series) using loop. Here's how:

Example 1:

$S = 1 + 2 + 3 + 4 + 5$

$$+ + \sum_{i=1}^5 i$$

we need to create a variable to store the sum (say s)

```
int s=0;  
for(int i=1; i<=5; i++){  
s+= i;  
}
```

```
System.out.println(s);
```

Example 2:

$$S = \frac{1}{1} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \frac{1}{9} + \frac{1}{11} + \frac{1}{13}$$

$$+= 2 \sum_{i=1}^{13} 1/i$$

```
int s=0;  
for(int i=1; i<=13; i+=2){  
s+= 1/i;  
}  
System.out.println(s);
```

**** In Series, always print S outside loop block**